> https://ac.nowcoder.com/acm/contest/95334

目前包括ABCDEFGHJKM

~~太菜了比赛的时候只做出来了8题~~

个人难度排序 $\mathrm{A} < \mathrm{B} < \mathrm{F} < \mathrm{G} < \mathrm{J} < \mathrm{K} < \mathrm{D} < \mathrm{H} < \mathrm{C} < \mathrm{E} < \mathrm{M}$

I L钛难还没补
签到题：A B
简单：F G J
中等： K D H C
困难： E M

除困难题外代码给出python样例（困难题cpp），实在看不懂拿AI翻译一下

签到题

A.一起奏响历史之音！

> https://ac.nowcoder.com/acm/contest/95334/A

在中国传统五声音调中，“宫、商、角、徵、羽”（读音为gōng shāng jué zhǐ yǔ）分别类似现在简谱中的1、2、3、5、6。即宫等于1(Do)，商等于2(Re)，角等于3(Mi)，徵等于5(Sol)，羽等于6(La)。

现在，牛可乐从历史的尘埃中找到了一个音节序列，他想请你帮他判断，检查序列是否由五声音调中的部分或全部音调组成。

题解

遍历数组，判断是否每个数都是1、2、3、5、6中的1个即可

时间复杂度 $O(n)$

```
a = list(map(int, input().split()))
for i in a:
    if i not in (1, 2, 3, 5, 6):
        print('NO')
        break
else:
    print('YES')
```

B. 能去你家蹭口饭吃吗

> https://ac.nowcoder.com/acm/contest/95334/B

牛妹家里有 n 个碗，第 i 个碗的容量为 a_i。

牛妹要求，牛可乐自带的碗至少要比牛妹家一半数量的碗容量更小，而为了能装更多的饭，牛可乐想要带尽可能大的碗。请你帮助牛可乐计算，他最大能带多大的碗。

题解

数组的中位数满足一半得数小于它，一半的数大于他。

我们只需取 $a_{\lfloor \frac{n}{2} \rfloor} - 1$(索引从0开始)，这样至少有一半数比它大。

时间复杂度 $O(n \cdot \log n)$

```
n = int(input())
a = list(map(int, input().split()))
a.sort()
print(a[n//2]-1)
```

简单题

F.一起找神秘的数！

> https://ac.nowcoder.com/acm/contest/95334/F

对于给定的区间 l 到 r，从中选取两个整数 x 和 y，使得下式成立：

$$x + y = (x \text{ or } y) + (x \text{ and } y) + (x \text{ xor } y)$$

求解有多少对不同的满足条件的 x, y。

题解

~~比赛的时候没证出来，看见大家库库交猜了一发过了~~
对于任意一位二进制有
$(1\&1) + (1|1) = 2 = 1 + 1$
$(1\&0) + (1|0) = 1 = 1 + 0$
$(0\&1) + (0|1) = 1 = 0 + 1$
$(0\&0) + (0|0) = 0 = 0 + 0$
所以 $x + y = (x \text{ or } y) + (x \text{ and } y)$

即求满足条件 $x \text{ xor } y = 0$ 的 (x, y)

即 $x = y$ 满足条件

时间复杂度 $O(1)$

```
t = int(input())
for _ in range(t):
    l, r = map(int, input().split())
    print(r-l+1)
```

G. 一起铸最好的剑！

> https://ac.nowcoder.com/acm/contest/95334/G

牛可乐从古籍中得知，铸剑的温度越接近 n 度，剑的品质越好。现在，他正在研究他家的铸剑炉，想要铸出全村最好的剑。

启动炉子时，牛可乐已经添过了一次柴，所以铸剑炉的初始温度为 m 度。此后，牛可乐每次添柴可以使得铸剑炉的温度提高到原来的 m 倍，即温度变为 $m^2, m^3, \cdots$。

牛可乐想要知道，他最少需要添多少次柴（包括启动炉子时添的那一次），才能使得铸剑炉的温度最接近 n 度，这样他就能铸出一把品质最好的剑。

题解

当 $m = 1$ 时，每次添柴不会增加温度，答案为1

当 $m >= \lceil \sqrt{2 \cdot 10^9} \rceil$ 时，操作一次必定会使温度远离 n ，答案为1

其余情况下，令 $k = \log m^n$, 我们比较 m^k 和 m^{k+1} 哪个接近G， 输出对应的次数

时间复杂度 $O(\log n)$

```
def fast_power(a, b):
    result = 1
    while b > 0:
        if b % 2 == 1:  # 如果当前位是 1
            result *= a
        a *= a  # 将 a 自乘
        b //= 2  # 右移一位
    return result


from math import log
t = int(input())
for _ in range(t):
    n, m = map(int, input().split())
    if m >= n:
        print(1)
    elif m >= 44722:
        print(1)
    elif m == 1:
        print(1)
    else:
        k = int(log(n, m))
        l = fast_power(m, k)
        r = fast_power(m, k+1)
        if abs(n - l) <= abs(r - n):
            print(k)
        else:
            print(k+1)
```

J. 数据时间?

> https://ac.nowcoder.com/acm/contest/95334/J

~~何で春日影にやっだの？！~~

牛可乐正在分析最近几个月用户登录牛客 APP 的情况，他所收集的数据由三个字段构成，分别为：user_id 表示用户 ID，login_date 表示登录日期，login_time 表示登录时间

由于牛可乐同时还在忙着演奏■■■、去牛妹家吃一勺饭、钻研可爱的字符串.....所以他想请你帮助他统计 h 年 m 月份通勤、午休、临睡这三个时段各有多少人登录过 APP。特别地，同一个人在同一时段多次登录视作一次。

在本题中，通勤时间段为 07:00:00-09:00:00 和 18:00:00-20:00:00；午休时间段为 11:00:00-13:00:00；临睡时间段为 22:00:00-01:00:00。时间段均包含左右边界值。

样例2
输入

2 2025 1
1000 2024-12-31 23:59:59
1000 2025-01-01 00:00:00

输出

0 0 1

说明

先判定年份与月份，再判定时间。

题解

注意题目中样例2的说明

~~比赛的时候成功被一坨的题干给迷惑了~~

模拟题

对于每个符合年月时间，我们先判断其处于哪个时间段内，再判断有没有相同用户同一时间段的登录记录即可（用map或set维护）

时间复杂度 $O(n)$

```
from collections import defaultdict

def trans_time(s):
    s = list(map(int, s.split(':')))
    return s[0] * 3600 + s[1] * 60 + s[2]

tongqingtt1 = [trans_time('07:00:00'), trans_time('09:00:00')]
tongqingtt2 = [trans_time('18:00:00'), trans_time('20:00:00')]
wushuitt = [trans_time('11:00:00'), trans_time('13:00:00')]
linshuitt1 = [trans_time('22:00:00'), trans_time('23:59:59')]
linshuitt2 = [trans_time('00:00:00'), trans_time('01:00:00')]

def get_type(tt):
    if tongqingtt1[0] <= tt <= tongqingtt1[1]:
        return 0
    if tongqingtt2[0] <= tt <= tongqingtt2[1]:
        return 0
    if wushuitt[0] <= tt <= wushuitt[1]:
        return 1
    if linshuitt1[0] <= tt <= linshuitt1[1]:
        return 2
    if linshuitt2[0] <= tt <= linshuitt2[1]:
        return 2
    return 3

n, h, m = map(int, input().split())
ans = [0, 0, 0]
memo = [defaultdict(int) for i in range(3)]
for _ in range(n):
    uid, da, t = input().split()
```

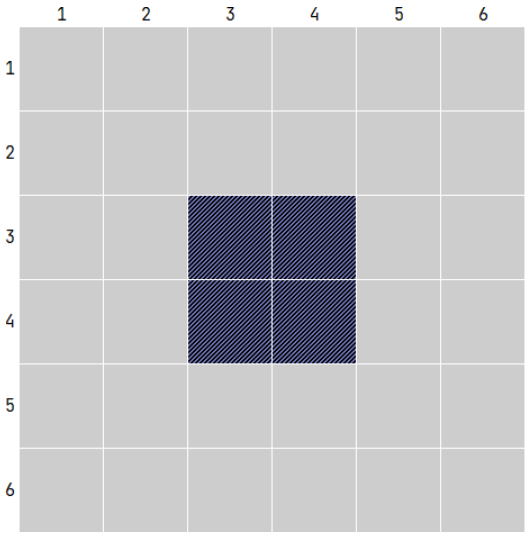
```
    uid = int(uid)
    y, mo, d = map(int, da.split('-'))
    if y == h and m == mo:
        tt = trans_time(t)
        tttype = get_type(tt)
        if tttype != 3:
            if not memo[tttype][uid]:
                ans[tttype] += 1
                memo[tttype][uid] = 1
print(*ans)
```

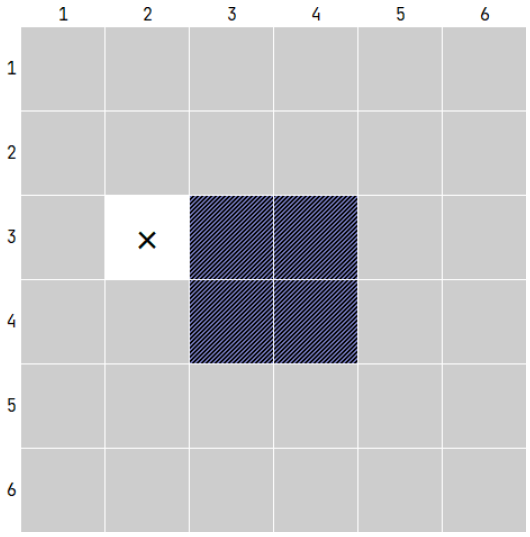
中等题

K.可以分开吗?

> https://ac.nowcoder.com/acm/contest/95334/K

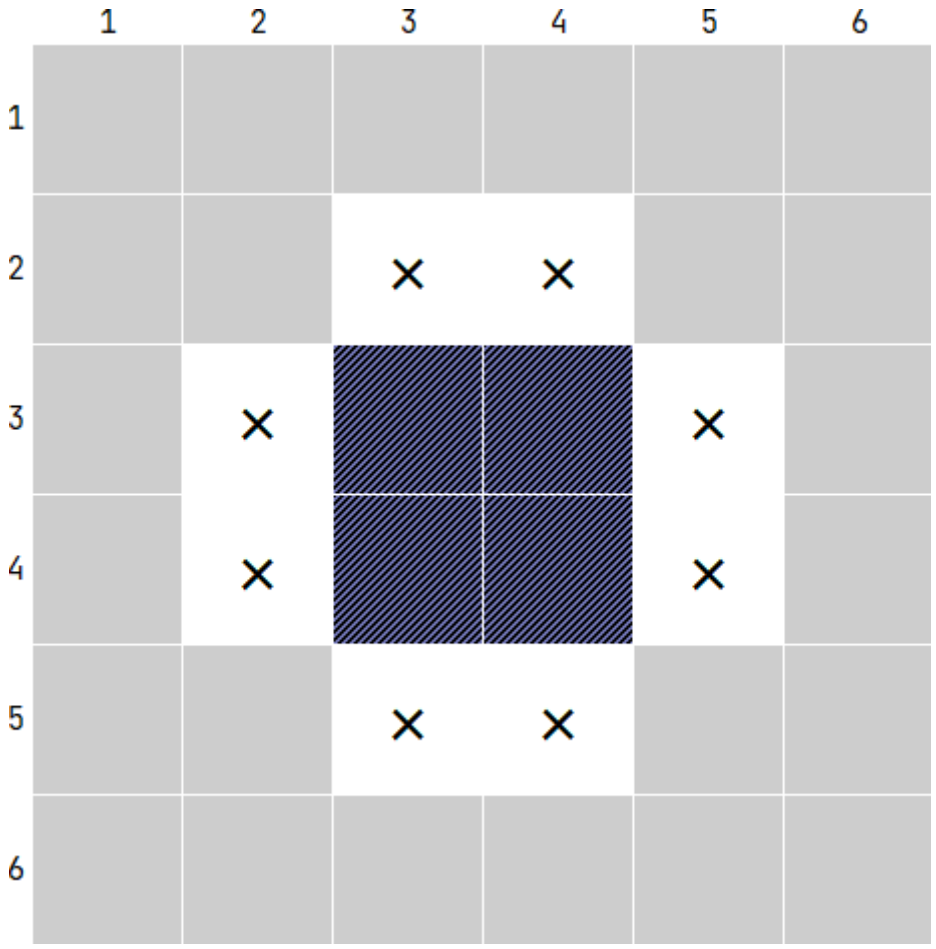
牛可乐有 $n \times m$ 个砖块排布成 n 行 m 列的矩阵，我们使用 (i, j) 表示矩阵中从上往下数第 i 行和从左往右数第 j 列的砖块。如下左图就是一个 6 行 6 列的矩阵。有的砖块能被敲碎，为了便于辨认，我们使用灰色绘制它们；有的砖块不能被敲碎，我们使用蓝色斜线绘制它们。如下右图所示，第 3 行第 2 列的砖块 $(3, 2)$ 是灰色的，能被敲碎，使用一个叉代表其被敲碎了。



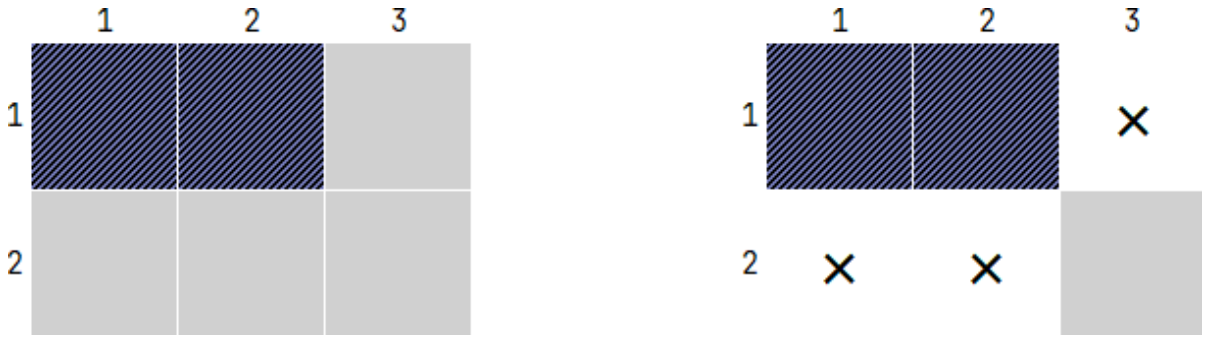


对于使用蓝色斜线绘制的砖块，它们是一体的，牛可乐定义“蓝色极大连通块”为最大化上下左右（即四连通）连接的蓝色砖块个数、且不包含灰色砖块的连通块，例如，在上图中，唯一的“蓝色极大连通块”包含 4 个蓝色砖块，分别为 $(3, 3), (3, 4), (4, 3), (4, 4)$。

与真实生活一样，如果一个“蓝色极大连通块”的任意一个砖块都不与灰色砖块存在共边，则认为这个“蓝色极大连通块”完好地从原砖块掉落。例如，在上图中，要使得“蓝色极大连通块”掉落，至少需要敲碎八块砖块，如下图所示。



特别地，对于位于边界上的“蓝色极大连通块”，不需要关注悬空的那些。例如，在下左图中，唯一的“蓝色极大连通块”包含 2 个蓝色砖块，分别为 $(1,1),(1,2)$，其位于边界上，要使得它掉落，至少需要敲碎 3 块砖块，如下图所示。



现在，对于牛可乐给你的砖块矩阵，你需要帮助他分离出任意一块“蓝色极大连通块”，计算最少需要敲碎的灰色砖块个数。

题解

首先可以使用并查集/BFS/DFS获取所有的连通块。

对每个连通块，我们遍历其中每个点，统计点的四个方向上有多少个灰色砖块（注意去重）。

取所有中的最小值即可

时间复杂度 $O(n \cdot m \cdot \log(n \cdot m))$

```
class DSU:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [1] * n
```

```python
    def find(self, p):
        if self.parent[p] != p:
            self.parent[p] = self.find(self.parent[p])  # 路径压缩
        return self.parent[p]

    def union(self, p, q):
        rootP = self.find(p)
        rootQ = self.find(q)

        if rootP != rootQ:
            # 按秩合并
            if self.rank[rootP] > self.rank[rootQ]:
                self.parent[rootQ] = rootP
            elif self.rank[rootP] < self.rank[rootQ]:
                self.parent[rootP] = rootQ
            else:
                self.parent[rootQ] = rootP
                self.rank[rootP] += 1

    def get_all_sets(self):
        sets = {}
        for i in range(len(self.parent)):
            root = self.find(i)
            if root not in sets:
                sets[root] = []
            sets[root].append(i)
        return sets


n, m = map(int, input().split())
maze = [list(map(int, list(input()))) for i in range(n)]

d = [[-1, 0], [1, 0], [0, 1], [0, -1]]
dsu = DSU(n*m)
for x in range(n):
    for y in range(m):
        if maze[x][y]:
            for dx, dy in d:
                nx, ny = x+dx, y + dy
                if 0 <= nx  < n and 0 <= ny  < m  and maze[nx][ny] == 1:
                    dsu.union(x*m+y, nx*m+ny)
ans = n * m
for points in dsu.get_all_sets().values():
    need = set()
    f = 1
    for k in points:
        x0 = k//m
        y0 = k % m
        if not maze[x0][y0]:
            f = 0
            break
        for dx, dy in d:
            nx, ny = x0+dx, y0 + dy
            if 0 <= nx  < n and 0 <= ny  < m  and maze[nx][ny] == 0:
                need.add((nx, ny))
```

```
    if f:
        ans = min(ans, len(need))
print(ans)
```

D.字符串里串

> https://ac.nowcoder.com/acm/contest/95334/D

牛可乐定义字符串 s 的可爱度 k 为这样的一个最大整数，使得存在长度为 k 的连续子串[1] a、长度为 k 的不连续子序列[2] b，满足 $a=b$。特别地，若不存在符合要求的 a,b，则可爱度为 0。

现在，对于给定的字符串 s，求解其可爱度。

子串[1]为从原字符串中，连续的选择一段字符（可以全选、可以不选）得到的新字符串。

仅在本题中，不连续子序列[2]定义为至少由两段不相邻的**非空**子串构成。

题解

为了最大化长度，我们可以贪心的选择左端点在 1 或 右端点在 n。

（假如我们选择的区间不满足上述条件，我们一定可以左移左端点或右移右端点，这不会影响 $a=b$）

以左端点在 1 为例，我们可以从右端点为 n 时开始向左扫描，当右端点对应的字符在右边出现过时，此时可以构造一对 (a,b)，记录答案。

再反方向扫描一遍即可。

时间复杂度 $O(n)$

```
from collections import defaultdict
n = int(input())
s = input()
f1 = 0
ans1 = 0
memo1 = defaultdict(int)
for i in range(n-1, -1, -1):
    if i != n-1 and memo1[s[i]]:
        ans1 = max(ans1, i+1)
    memo1[s[i]] = 1
ans2 = 0
memo2 = defaultdict(int)
for i in range(n):
    if i and memo2[s[i]]:
        ans2 = max(ans2, n-i)
    memo2[s[i]] = 1
print(max(ans1, ans2))
```

H. 一起画很大的圆！

> https://ac.nowcoder.com/acm/contest/95334/H

牛可乐正在研究二维平面。现在，他已经划定了一个矩形区域，左侧边界为 $x=a$，右侧边界为 $x=b$，下侧边界为 $y=c$，上侧边界为 $y=d$，他想要在四条直线所围成的矩形的**边界上**找到三个不同的整数点 A,B,C，使得过这三个点画出的圆半径最大。

请你帮助他实现！

题解

不妨先假设 $b-a>d-c$ (另一种情况转 90° 即可。

由正弦定理 $2R=\frac{|AB|}{sin\angle ACB}=\frac{|AC|}{sin\angle ABC}=\frac{|BC|}{sin\angle BAC}$

为了最大化最大角，我们肯定要让三个点尽可能在一条直线上。

有两种思路

1）选取 $(a,c),(b,d),(b-1,d)$,此时最靠近对角线

2）选取 $(a,c),(a+1,c),(b,c+1)$,此时最靠近长边

3）选取 $(a,c),(a,c+1),(a+1,d)$,此时最靠近短边

计算可知 2) 的半径最大

时间复杂度 $O(1)$

```
t = int(input())
for _ in range(t):
    a, b, c, d = map(int, input().split())
    if b - a >= d - c:
        print(a, c)
        print(a+1, c)
        print(b, c+1)
    else:
        print(a, c)
        print(a, c+1)
        print(a+1, d)
```

C.字符串外串

> https://ac.nowcoder.com/acm/contest/95334/C

牛可乐定义字符串 s 的可爱度为最大的整数 k，满足存在一个长度为 k 的连续子串 [1] a、和一个长度为 k 的不连续子序列 [2] b，满足 $a=b$。

现在，牛可乐给定你两个整数 n,m，并且询问你是否存在长度为 n、仅由小写字母构成的字符串 t ，使得 t 的可爱度恰好等于 m。如果存在，输出任意一个符合条件的字符串 t。

子串 [1] 为从原字符串中，连续的选择一段字符（可以全选、可以不选）得到的新字符串。

仅在本题中，不连续子序列 [2] 定义为至少由两段不相邻的**非空**子串构成。

题解

题目要求不连续子序列 [2] 定义为至少由两段不相邻的**非空**子串构成，因此 $n=m$ 时一定无法构造。

由D可知，可爱度取决于从左到右或从右到左第一次有两个重复字符出现

不妨构造一个左右尽可能对称的字符串便于讨论

当$m*2>=n$ 时，我们只需要让边上 $n-m$个字符保持不同（左右对称），中间的 $n-2*m$个字符取两侧出现过的即可

当$m*2<n$ 时，我们还需要确保中间 $n-2*m$个字符与边上 $n-m$个字符保持不同，所以 必须满足 $n-2*m+m=n-m<=26$,否则无解。

时间复杂度 $O(n)$

```
alpha = 'abcdefghijklmnopqrstuvwxyz'
t = int(input())
for _ in range(t):
    n, m = map(int, input().split())
    if n - m > 26:
        print('NO')
    elif n == m:
        print('NO')
    elif n >= m * 2:
        print('YES')
        print(alpha[:m] + alpha[m:m+ n- 2 * m] + alpha[:m][::-1])
    else:
        s = alpha[:n-m]
        s += 'a' * (n - len(s)*2)
        s += alpha[:n-m][::-1]
        print('YES')
        print(s)
```

困难题

E.一起走很长的路！

> https://ac.nowcoder.com/acm/contest/95334/E

牛妹正在玩多米诺骨牌。她将 n 张多米诺骨牌从左到右排成一排，每一张多米诺骨牌都有重量，第 i 张多米诺骨牌的重量记为 a_i。随后，牛妹定义了新的游戏规则，初始时她会选定 $l, r\ (1 \leqq l \leqq r \leqq n)$ ，一轮游戏过程如下：

- 手动选择第 l 张多米诺骨牌，将其向右推倒；
- 对于第 $l+1$ 张多米诺骨牌，若满足 $a_l \geqq a_{l+1}$ ，则其也会向右倒下；
- 对于第 $l+2$ 张多米诺骨牌，若满足 $a_l + a_{l+1} \geqq a_{l+2}$ ，则其也会向右倒下；
- .....；
- 对于第 i 张多米诺骨牌，若其左侧的第 $i-1$ 张多米诺骨牌倒下，且第 i 张多米诺骨牌的重量不大于其左侧倒下的全部多米诺骨牌的重量之和，则其也向右倒下，即当 $\sum_{k=l}^{i-1} a_k \geq a_i$ 时，其向右倒下；
- 对于第 r 张多米诺骨牌，如果其倒下了，那么这一局游戏就是完美的。

致力于成为牛客星球技术大牛的牛可乐希望牛妹选择的每一对 l, r 都能使得这一局游戏是完美的。为此，在牛妹选定 l, r 后，牛可乐会调整多米诺骨牌的重量。具体地，每一次调整他会选择一块多米诺骨牌，然后将其重量增加 1 或者减少 1。

快乐是牛妹的，你需要帮助牛可乐计算，对于每一对 l, r，牛可乐至少需要调整多少次多米诺骨牌的重量，才能使得这一局游戏是完美的。调整仅针对当前轮游戏，在每一轮游戏结束后骨牌的重量都会恢复。

题解

定义 $pre_i = \sum_{j=1}^{i} a_j$

对于位置 i, 其左边需要增加的块数为 $max(0, a_i - \sum_{j=l}^{i-1} a_j)$ 。

显然我们可以把所有的更改都放在 l上。

则答案为 $max(0, max(j \in [l+1, r], a_i - \sum_{j=l+1}^{i-1} a_j))$(最左边的那一块不需要随着前一块倒下)。

而$\sum_{j=l+1}^{i-1} a_j = pre_{i-1} - pre_{l-1}$

所以可以变为 $max(0, max(j \in [l+1, r], a_i - pre_{i-1}) + pre_{l-1})$

而 $max(j \in [l+1, r], a_i - pre_{i-1})$可用ST表/线段树快速得到

注意 $l = r$ 时答案为 0

时间复杂度 $O(n \cdot logn + q)$ 或 $O(n \cdot logn + q \cdot logn)$

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

inline ll read(){
    ll s = 0, w = 1; char ch = getchar();
    while (ch < 48 || ch > 57) { if (ch == '-') w = -1; ch = getchar(); }
    while (ch >= 48 && ch <= 57) s = (s << 1) + (s << 3) + (ch ^ 48), ch =
getchar();
    return s * w;
}
inline void pt(ll x){if(x<0) putchar('-'),x=-x;if(x>9)
pt(x/10);putchar(x%10+'0');}

void print(ll x){pt(x), puts("");}

const ll MAXN = 200009;
ll a[MAXN], LOG2[MAXN], dpmax[MAXN][22];
ll presum[MAXN];
int main(){
    LOG2[0] = -1;
    for(ll i=1;i<MAXN;i++){
        LOG2[i] = LOG2[i>>1] + 1;
    }
    ll n = read(), q = read();
    for(ll i=1;i<=n;i++){
        a[i] = read();
        presum[i] = presum[i-1] + a[i];
        dpmax[i][0] = a[i] - presum[i-1];
    }
    ll p = LOG2[n];

    for(ll k=1;k<=p;k++){
        for(ll s=1;s+(1<<k)<=n+1;s++){
            dpmax[s][k] = max(dpmax[s][k-1], dpmax[s+(1<<(k-1))][k-1]);
        }
    }
    auto query = [&](ll l, ll r){
        ll k = LOG2[r-l+1];
        return max(dpmax[l][k], dpmax[r-(1<<k)+1][k]);
    };
    while(q--){
        ll l = read(), r = read();
        if (l == r) print(0);
        else print(max(0ll, query(l+1, r) + presum[l-1]));
    }
```

```
}
```

M. 那是我们的影子

> https://ac.nowcoder.com/acm/contest/95334/M

由 $3 \times n$ 个单元格构成的 3 行 n 列异形数独规则如下：

- 每一个单元格都需要填入 1 到 9 之间的整数；
- 任意一个 3×3 的子矩阵中都不包含重复的数字；

现在，牛可乐已经填入了一些数字，请你在此基础上帮助他计算，这个异形数独一共可以构造出多少个不同的合法解。由于答案可能很大，请输出答案对 $10^9 + 7$ 取模后的结果。

题解

由于每个3×3 的子矩阵都要取一遍，第 1、 4、...列，第 2、 5、...列，第3、 6、...列可以取得数字的集合是相同的。

即 $i \, mod \, 3$ 相同的列可以取得数字的集合是相同的。

所以如果 $i \, mod \, 3$ 相同的列出现了三个以上不同的数字，必然会导致另外有一列无法取到三个数，此时无解。

若一列出现了两个相同的数字，无解。

如果一个数字出现在了 $i \, mod \, 3$ 不同的列，必定有一个 3×3 的子矩阵中都包含重复的数字，无解。

有解的情况下，首先考虑如何把没有出现过的数字分配到三种列中。

假设 $i \, mod \, 3 = j$ 的列已经出现了num_j个数，则此时这一列还可以选取的数为$a_j = 3 - num_j$

由组合得分配数字的方案数为 $C_{a_0+a_1+a2}^{a_0} \cdot C_{a_1+a2}^{a_1}$

对每一列，如果有 k 个问号，由排列得方案数为 A_k^k

由乘法原理，总方案数为$C_{a_0+a_1+a2}^{a_0} \cdot C_{a_1+a2}^{a_1} \cdot \prod_{i=1}^{n} A_{k_i}^{k_i}$

时间复杂度 $O(3 \cdot n)$

```
#include <bits/stdc++.h>
using namespace std;

typedef long long ll;
typedef __int128 INT;
typedef pair<long long,long long> PLL;
typedef tuple<ll,ll,ll> TLLL;
const ll inf =  0x3f3f3f3f;
const ll INF = INT_MAX;

inline ll read(){
    ll s = 0, w = 1; char ch = getchar();
    while (ch < 48 || ch > 57) { if (ch == '-') w = -1; ch = getchar(); }
    while (ch >= 48 && ch <= 57) s = (s << 1) + (s << 3) + (ch ^ 48), ch =
getchar();
    return s * w;
}
```

```
inline void pt(ll x){if(x<0) putchar('-'),x=-x;if(x>9)
pt(x/10);putchar(x%10+'0');}

void print(ll x){pt(x), puts("");}

const ll mod = 1e9+7;
ll fac[100009];
ll ifac[100009];
ll ksm(ll a,ll b){
    ll ans = 1;
    while(b){
        if(b&1)ans = ans * a % mod;
        a = a * a % mod;
        b >>= 1;
    }
    return ans;
}
ll C(ll a,ll b){
    return fac[a] * ifac[b] % mod * ifac[a-b] % mod;
}

ll A(ll n, ll m){
    return fac[n] * C(n, m) % mod;
}

void solve(){
    ll n = read();
    string s[4];
    for(ll i = 1;i<=3;i++){
        cin>>s[i];
        s[i] = '&' + s[i];
    }
    for(ll i = 1;i<=n;i++){
        map<ll, ll> memo;
        for(ll j = 1;j<=3;j++){
            if (s[j][i] == '?'){
                continue;
            }
            if(memo[s[j][i]-'0']){
                print(0);
                return;
            }
            memo[s[j][i]-'0'] = 1;
        }
    }
    set<ll> appear[3];
    for (ll i = 1;i<=n;i++){
        for(ll j=1;j<=3;j++){
            if (s[j][i] == '?'){
                continue;
            }
            s[j][i] -= '0';
            appear[i%3].insert(s[j][i]);
        }
    }
    for(ll i=0;i<3;i++){
```

```
        if(appear[i].size() > 3){
            print(0);
            return;
        }
    }
    for(ll num=1;num<=9;num++){
        ll has = 0;
        for(ll i=0;i<3;i++){
            if (appear[i].find(num)!=appear[i].end()){
                if (has){
                    print(0);
                    return;
                }
                has = 1;
            }
        }
    }
    vector<ll> f(n+1);
    for (ll i = 1;i<=n;i++){
        for(ll j=1;j<=3;j++){
            f[i] += s[j][i] == '?';
        }
    }
    ll empty = 0;
    for(ll i=0;i<3;i++){
        empty += (3 - appear[i].size());
    }
    ll ans = C(empty, (3 - (ll)appear[0].size())) * C(empty-(3 -
(ll)appear[0].size()), (3 - (ll)appear[1].size()))%mod;
    for(ll i=1;i<=n;i++){
        ans = (ans * A(f[i], f[i])) %mod;
    }
    print(ans);
}

int main(){
    fac[0] = ifac[0] = 1;
    for(int i = 1; i <= 100007; i++){
        fac[i] = fac[i-1] * i % mod;
    }
    ifac[100007] = ksm(fac[100007], mod-2);
    for(int i = 100007-1; i >= 1; i--){
        ifac[i] = ifac[i+1] * (i+1) % mod;
    }
    ll t = read();
    while(t--){
        solve();
    }
}
```